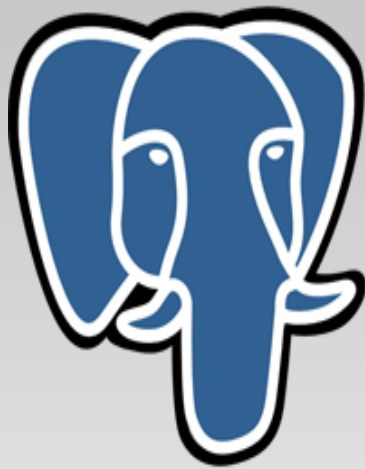




PostgreSQL

POSTGRES DATABASE ADMINISTRATION



STEPHEN NJOROGUE

LinkedIn: www.linkedin.com/in/stephen-njoroge

|SENIOR DATABASE ADMINISTRATOR|ORACLE| POSTGRESQL | MSSQL|OCI|AWS|

CHAPTER 3: POSTGRESQL PHYSICAL STRUCTURES – CLONING A DATABASE.

About the Author:

I am **Stephen Njoroge**, a Senior Database Engineer with over 8+ years of expertise in designing, securing and optimizing high-performance database systems across the banking and fintech sectors. My core technical proficiency spans major databases: Oracle, PostgreSQL, MS SQL Server and MySQL-along with advanced scripting in SQL and Python for automation and data pipeline development. I architect and manage cloud and virtualization solutions on OCI, AWS, and Proxmox for onprem systems and implement comprehensive DevOps practices using monitoring tools like Prometheus and Grafana and IT service management via iTop. Further, I deliver advanced data analytics and visualization using Tableau, Power BI and Oracle BI. As a Certified Data Protection Officer, I integrate robust data protection, compliance and security measures including vulnerability assessment and role-based access control into every layer of the data infrastructure.

Cloning a Database in PostgreSQL

Using CREATE DATABASE with TEMPLATE

- Check Source Database

```
-- Connect to source database
```

```
\c trainingdb
```

```
-- Check database size and objects
```

```
SELECT
```

```
    pg_size_pretty(pg_database_size('trainingdb')) as size,  
    (SELECT COUNT(*) FROM pg_tables WHERE schemaname NOT IN ('pg_catalog', 'information_schema')) as  
table_count,  
    (SELECT COUNT(*) FROM pg_views WHERE schemaname NOT IN ('pg_catalog', 'information_schema')) as  
view_count;
```

```
trainingdb=# SELECT  
    pg_size_pretty(pg_database_size('trainingdb')) as size,  
    (SELECT COUNT(*) FROM pg_tables WHERE schemaname NOT IN ('pg_catalog', 'information_schema')) as table_count,  
    (SELECT COUNT(*) FROM pg_views WHERE schemaname NOT IN ('pg_catalog', 'information_schema')) as view_count;  
 size | table_count | view_count  
-----+-----+-----  
8030 kB |          4 |          1  
(1 row)  
  
trainingdb=#
```

```
-- List all objects
```

```
\dt -- Tables
```

```
\di -- Indexes
```

```
\df -- Functions
```

```
trainingdb=# \dt
      List of tables
 Schema |      Name      | Type | Owner
-----+-----+-----+-----
 training | courses        | table | postgres
 training | enrollments    | table | postgres
 training | trainees       | table | postgres
 training | trainers       | table | postgres
(4 rows)

trainingdb=# \di
      List of indexes
 Schema |      Name      | Type | Owner | Table
-----+-----+-----+-----+-----
 training | courses_pkey   | index | postgres | courses
 training | enrollments_pkey | index | postgres | enrollments
 training | idx_enrollments_trainee | index | postgres | enrollments
 training | idx_trainees_email | index | postgres | trainees
 training | trainees_email_key | index | postgres | trainees
 training | trainees_pkey  | index | postgres | trainees
 training | trainers_email_key | index | postgres | trainers
 training | trainers_pkey  | index | postgres | trainers
(8 rows)

trainingdb=# \df
      List of functions
 Schema |      Name      |      Result data type      | Argument data types | Type
-----+-----+-----+-----+-----
 training | get_trainee_courses | TABLE(course_name character varying, enrolled_on date) | p_trainee_id integer | func
(1 row)

trainingdb=#

\dv -- Views
```

```
trainingdb=# \dv
      List of views
 Schema |      Name      | Type | Owner
-----+-----+-----+-----
 training | vw_course_enrollments | view | postgres
(1 row)

trainingdb=#
```

Ensure No Active Connections

```
-- Check active connections to source database
```

```
SELECT
    pid,
    username,
    application_name,
    client_addr,
    state,
    query
FROM pg_stat_activity
WHERE datname = 'trainingdb'
AND pid <> pg_backend_pid();
```

```

trainingdb=# SELECT
    pid,
    username,
    application_name,
    client_addr,
    state,
    query
FROM pg_stat_activity
WHERE datname = 'trainingdb'
AND pid <> pg_backend_pid();
 pid | username | application_name | client_addr | state | query
-----+-----+-----+-----+-----+-----
(0 rows)

trainingdb=#

```

-- If there are connections, terminate them (if safe)

```

SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE datname = 'trainingdb'
AND pid <> pg_backend_pid();

```

```

trainingdb=# SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE datname = 'trainingdb'
AND pid <> pg_backend_pid();
 pg_terminate_backend
-----
(0 rows)

trainingdb=#

```

Create Clone Database

-- Connect to postgres database (or any database except the source)

```
\c postgres
```

-- Clone database using template (FASTEST METHOD)

```

CREATE DATABASE clonedb
    WITH
    TEMPLATE trainingdb
    OWNER = postgres;

```

```
postgres=# CREATE DATABASE clonedb
        WITH
        TEMPLATE trainingdb
        OWNER = postgres;
CREATE DATABASE
postgres=#
```

Using pg_dump & pg_restore

Backup Source Database

```
# Plain SQL format (portable, human-readable)
```

```
pg_dump -U postgres \
--clean --create --if-exists \
trainingdb > trainingdb_backup.sql
```

```
postgres@998-986-02:/home/steve$ pg_dump -U postgres \
--clean --create --if-exists \
trainingdb > trainingdb_backup.sql
postgres@998-986-02:/home/steve$ ls -lrt trainingdb_backup.sql
-rw-rw-r-- 1 postgres postgres 12090 Feb  2 19:27 trainingdb_backup.sql
postgres@998-986-02:/home/steve$
```

Create Empty Target Database

```
-- Connect to postgres database
\c postgres
```

```
-- Create empty database with same parameters
```

```
CREATE DATABASE clonedb
        WITH
        OWNER = postgres
        TEMPLATE = template0
        CONNECTION LIMIT = -1;
```

```

postgres=# \c postgres
You are now connected to database "postgres" as user "postgres".
postgres=# CREATE DATABASE clonedb
        WITH
        OWNER = postgres
        TEMPLATE = template0
        CONNECTION LIMIT = -1;
CREATE DATABASE
postgres=# |

```

-- Or copy parameters from source database

```

SELECT
    'CREATE DATABASE clonedb WITH ' ||
    'OWNER = ' || pg_get_userbyid(datdba) || ' ' ||
    'ENCODING = ' || pg_encoding_to_char(encoding) || ' ' ||
    'LC_COLLATE = ' || datcollate || ' ' ||
    'LC_CTYPE = ' || datctype || ' ' ||
    'TEMPLATE = template0 ' ||
    'CONNECTION LIMIT = ' || COALESCE(datconlimit::text, '-1') || ';' as create_command
FROM pg_database
WHERE datname = 'trainingdb';

```

```

-----
CREATE DATABASE clonedb WITH OWNER = postgres ENCODING = 'UTF8' LC_COLLATE = 'C.UTF-8' LC_CTYPE = 'C.UTF-8' TEMPLATE = template0 CON
NECTION LIMIT = -1;
(1 row)

~
~
~

```

Restore Backup

For SQL format backup

```
psql -U postgres -p 5432 -d clonedb < trainingdb_backup.sql
```

```

postgres@998-986-02:/home/steve$ psql -U postgres -p 5432 -d clonedb < trainingdb_backup.sql
SET
SET
SET
SET
SET
SET
SET
set_config
-----

(1 row)

SET
SET
SET
SET
DROP DATABASE
CREATE DATABASE
ALTER DATABASE
You are now connected to database "trainingdb" as user "postgres".
SET
SET

```

Dropping a Database in a Cluster

Before Dropping ANY Database

```

-- 1. CHECK WHAT YOU'RE DROPPING
SELECT
    datname,
    pg_size_pretty(pg_database_size(datname)) AS size,
    datcollate,
    datctype,
    pg_encoding_to_char(encoding) AS encoding,
    datconlimit,
    datistemplate,
    dataallowconn,
    pg_database_size(datname) AS size_bytes
FROM pg_database
ORDER BY size_bytes DESC;

```

datname	size	datcollate	datctype	encoding	datconnlimit	datistemplate	dataallowconn	size_bytes
db1	8817 kB	C.UTF-8	C.UTF-8	UTF8	-1	f	t	9028111
training_db_restored	8737 kB	C.UTF-8	C.UTF-8	UTF8	-1	f	t	8946191
postgres	8302 kB	C.UTF-8	C.UTF-8	UTF8	-1	f	t	8500927
training_db	8193 kB	C.UTF-8	C.UTF-8	UTF8	-1	f	t	8389135
trainingdb	7958 kB	C.UTF-8	C.UTF-8	UTF8	-1	f	t	8148671
cloned_db	7793 kB	C.UTF-8	C.UTF-8	UTF8	-1	f	t	7979535
template1	7742 kB	C.UTF-8	C.UTF-8	UTF8	-1	t	t	7927487
mydb	7742 kB	C.UTF-8	C.UTF-8	UTF8	-1	f	t	7927487
clonedb	7670 kB	C.UTF-8	C.UTF-8	UTF8	-1	f	t	7853759
template0	7670 kB	C.UTF-8	C.UTF-8	UTF8	-1	t	f	7853759
app_template	7657 kB	C.UTF-8	C.UTF-8	UTF8	-1	t	t	7840271
myapp_prod	7585 kB	C.UTF-8	C.UTF-8	UTF8	100	f	t	7766543
my_custom_template	7585 kB	C.UTF-8	C.UTF-8	UTF8	-1	t	t	7766543
newdb	7585 kB	C.UTF-8	C.UTF-8	UTF8	-1	f	t	7766543
maintenance_db	7585 kB	C.UTF-8	C.UTF-8	UTF8	-1	f	f	7766543
myapp_dev	7585 kB	C.UTF-8	C.UTF-8	UTF8	20	f	t	7766543
myapp_reporting	7585 kB	C.UTF-8	C.UTF-8	UTF8	-1	f	t	7766543
db2	7513 kB	C.UTF-8	C.UTF-8	UTF8	-1	f	t	7692815

(18 rows)

postgres=#

-- 2. CHECK CONNECTIONS TO THE DATABASE

SELECT

```

    pid,
    username,
    application_name,
    client_addr,
    client_port,
    backend_start,
    state,
    query

```

FROM pg_stat_activity

WHERE datname = 'mydb'

ORDER BY backend_start;

postgres=# -- 2. CHECK CONNECTIONS TO THE DATABASE

SELECT

```

    pid,
    username,
    application_name,
    client_addr,
    client_port,
    backend_start,
    state,
    query

```

FROM pg_stat_activity

WHERE datname = 'mydb'

ORDER BY backend_start;

pid	username	application_name	client_addr	client_port	backend_start	state	query
5594	postgres	psql		-1	2026-02-02 19:41:55.890521+03	idle	

(1 row)

postgres=#

```
-- 3. CHECK IF DATABASE IS A TEMPLATE
```

```
SELECT datname, datistemplate
FROM pg_database
WHERE datname = 'clonedb';
```

```
postgres=# \l
postgres=# SELECT datname, datistemplate
FROM pg_database
WHERE datname = 'clonedb';
 datname | datistemplate 
-----+-----
 clonedb | f
(1 row)

postgres=#
```

Database Drop Commands:

```
-- Basic drop command
```

```
DROP DATABASE mydb;
```

```
postgres=# DROP DATABASE mydb;
DROP DATABASE
postgres=# |
```

Drop database with force

```
DROP DATABASE newdb WITH (FORCE);
```

```
postgres=# DROP DATABASE newdb WITH (FORCE);
DROP DATABASE
postgres=# |
```

Drop Multiple Databases

```
-- Drop multiple databases
```

```
for db in db1 db2; do
    psql -U postgres -c "DROP DATABASE IF EXISTS $db;"
done
```

```
postgres@998-986-02:/home/steve$ for db in db1 db2; do
  psql -U postgres -c "DROP DATABASE IF EXISTS $db;"
done
DROP DATABASE
DROP DATABASE
postgres@998-986-02:/home/steve$ |
```

Handling Active Connections

Using FORCE Option

```
-- This terminates connections automatically
DROP DATABASE maintenance_db WITH (FORCE);
```

```
postgres=# DROP DATABASE maintenance_db WITH (FORCE);
DROP DATABASE
postgres=#
```

Manual Connection Termination

```
-- Step 1: Prevent new connections
ALTER DATABASE training_db_restored ALLOW_CONNECTIONS false;
```

```
postgres=# ALTER DATABASE training_db_restored ALLOW_CONNECTIONS false;
ALTER DATABASE
postgres=#
```